

KAIST DSIL

Device-to-System Integration Laboratory for AI Hardware

Python-based Control of Semi-auto Probe Station

HEESOO YANG

Advisor: Prof. Jimin Kwon

kristron@unist.ac.kr

Department of Electrical Engineering,

Ulsan National Institute of Science and Technology (UNIST)

Today's Agenda

I. Overview

- System
- Software & control

II. Parameterizing Measurement Functions

- 전체 측정 과정의 구성
- Parameterized Measurement Function의 구성
- UI/UX

III. Manual Measurement vs. Code-based Measurement

- Results and discussion
- Problems and possible reasons

IV. Future Works

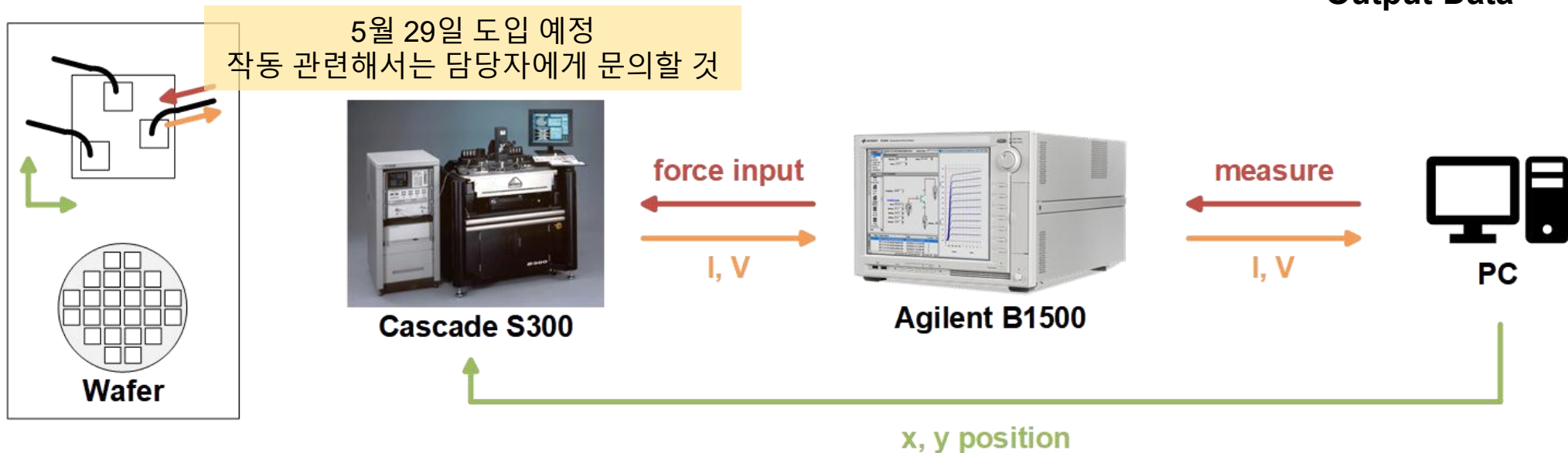
Overview

● System overview

- ✓ 사용 장비
 - 여기서 Cascade S300을 Semi-auto probe station, 세미오토 라고 부름
 - Cascade S300 : 웨이퍼 단위의 소자를 자동으로 컨택하는 probe station
 - Agilent B1500 : 전압·전류 특성을 측정하는 parameter analyzer
- ✓ 기존의 제어 방식
 - B1500은 그냥 측정장비라고 생각, 4200A도 비슷함
 - WaferPro Express 활용 : Keysight사에서 개발한 자동 측정 소프트웨어
- ✓ 한계: 측정 유연성 부족, 다중 장비 동기화 한계, 부정확한 소자 성능 판별, 비용

	A	B	C	D	E
		SMU1 Current Measurement (A)	SMU2 Current Measurement (A)	SMU3 Current Measurement (A)	SMU1 Voltage Output (V)
1					
2	0.00E+00	2.00E-11	0.00E+00	0.00E+00	-3.00E+00
3	1.00E+00	0.00E+00	0.00E+00	0.00E+00	-2.00E+00
4	2.00E+00	-2.00E-11	0.00E+00	0.00E+00	-2.00E+00
5	3.00E+00	2.00E-11	0.00E+00	2.00E-11	-1.00E+00
6	4.00E+00	-2.00E-11	0.00E+00	-2.00E-11	-6.00E-01
7	5.00E+00	2.00E-11	0.00E+00	0.00E+00	0.00E+00
8	6.00E+00	0.00E+00	0.00E+00	0.00E+00	6.00E-01
9	7.00E+00	2.00E-11	0.00E+00	0.00E+00	1.00E+00

Output Data



Overview

● Software & control

SCPI는 장비들을 제어하는 용도로 사용되는 명령어.

예를 들어, *IDN? 이라고 보내면 'Cascade Microtech, S300 Theta, 608820505, 3, 3'를 회신하는 식입니다.

- ✓ Solution: 외부 Desktop에서 **SCPI (Standard Commands for Programmable Instruments)** 명령어로 제어

→ Python을 통해 sequence 생성, 장비 제어 및 데이터 수집

- ✓ 전체 흐름

1. 측정할 소자 위치로 probe 이동 (S300)

2. 측정 설정값을 B1500에 전송

3. 측정 결과를 수신하고 저장

- ✓ 제어 구조

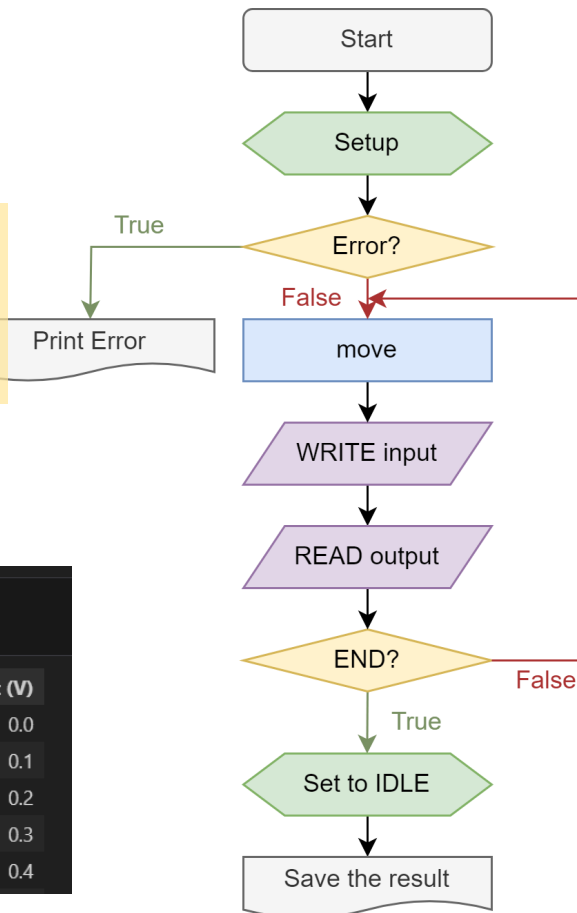
• 장비 상태 확인 → 이동 → 측정 → 결과 수집 → ... → 데이터 출력

최초에 자리를 잡는 건 수동으로 진행합니다.

하나를 먼저 찍어놓고, 그걸 바탕으로

좌표평면에 맞게 움직이고 찍는 걸

알아서 반복해주는 시스템이라고 생각하면 됩니다.



<pre># read measurement data all at once b1500.check_idle() #wait until measurement is finished data = b1500.read_data(2*nop) #Fac'</pre>					
[21] c:\Users\UNU_microscope\Heesoo_test\py					
data = data.applymap(self.data form					
c:\Users\UNU_microscope\Heesoo_test\py					
heads = data.iloc[[0]].applymap(lamb					
c:\Users\UNU_microscope\Heesoo_test\py					
data = data.applymap(lambda x: x[3])					
[22]					
data					
SMU1 Current Measurement (A) SMU2 Current Measurement (A) SMU3 Current Measurement (A) SMU1 Voltage Output (V)					
0 0.000000e+00 0.000000e+00 0.000000e+00 0.000000e+00 0.0					
1 0.000000e+00 -2.000000e-11 0.000000e+00 0.000000e+00 0.1					
2 0.000000e+00 0.000000e+00 0.000000e+00 0.000000e+00 0.2					
3 0.000000e+00 0.000000e+00 0.000000e+00 0.000000e+00 0.3					
4 2.000000e-11 0.000000e+00 0.000000e+00 0.000000e+00 0.4					
[22]					
SMU1 Current Measurement (A) SMI					
0 0.000000e+00					
1 0.000000e+00					
2 0.000000e+00					
3 0.000000e+00 0.000000e+00 0.000000e+00 0.3					
4 2.000000e-11 0.000000e+00 0.000000e+00 0.4					

Parameterizing Measurement Functions

● 전체 측정 과정의 구성

이건 제가 짠 코드의 flow라 동작 원리를 더 뜯어보고 고쳐도 돼요

✓ Setup

- B1500 GPIB 연결 여부 확인
- 연결되어 있는 UNIT 확인 (HRSMU 3개, WGFMU 1개 인식됨)
- Output Format 설정 (FMT21)
- Input Format (Measurement Mode) 설정

- SMU 초기화
- Sweep 범위, 단위 설정

장비를 동작시킬 때에는
장비에 무리가 가지 않도록 하는 게 중요합니다.

✓ Measurement

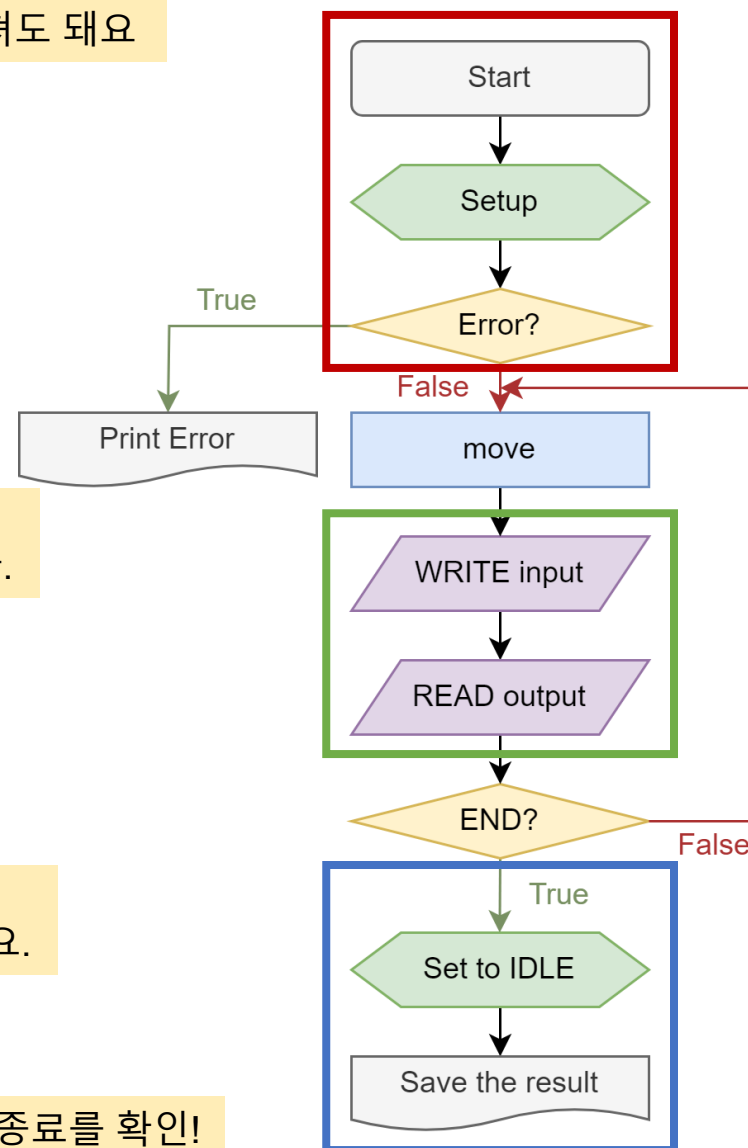
- Error 확인, Buffer 및 Timer clear
- Trigger 전송
- IDLE로 돌아올 때까지 대기
- Read Data

반드시 초기화를 확인하고,
각 단계가 끝날 때 장비의 IDLE을 확인해 주세요.

✓ Save & Initialize

- Data를 CSV로 저장, 각 SMU 초기화 및 장비 IDLE 확인

끝낼 때도 정상 종료를 확인!



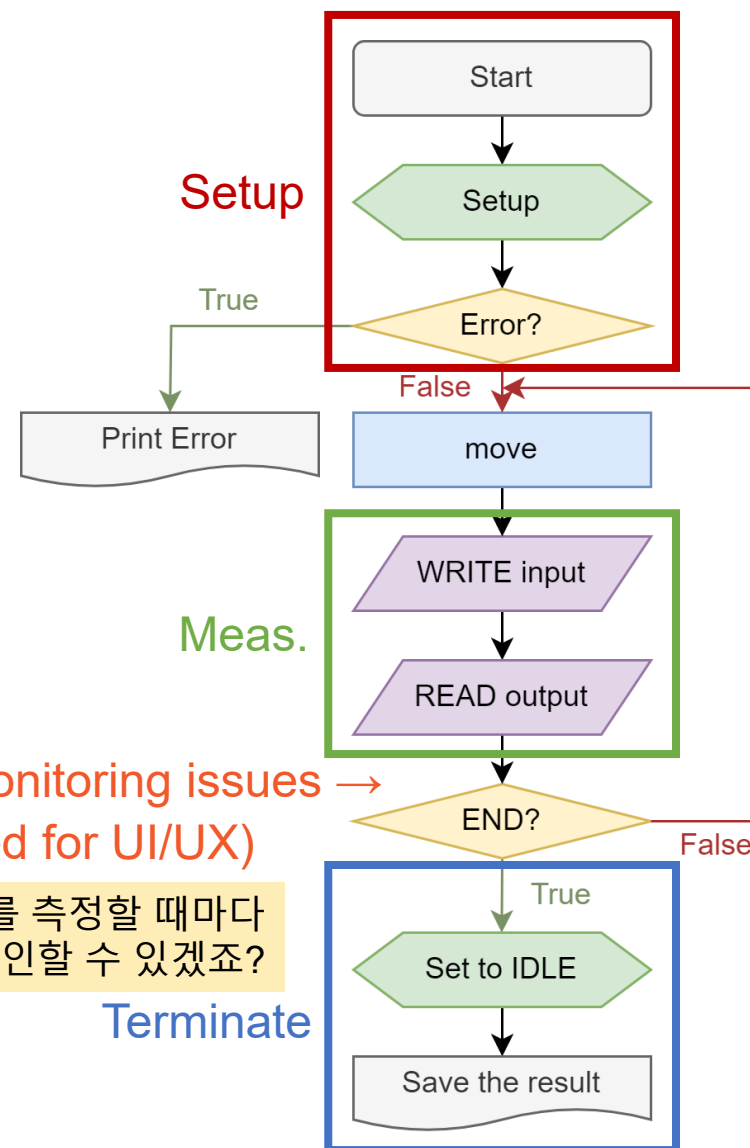
Parameterizing Measurement Functions

● Parameterized Measurement Function의 구성

✓ Parameterized Functions

- setup
 - INPUT : b1500
 - Function : 장비의 초기화 진행
- measurement
 - INPUT : b1500, vgs, vds, source_range(optional)
 - OUTPUT : data (각 SMU의 voltage와 current를 포함하는 array)
 - Function : 장비를 제어하여 SMU1과 SMU2에 voltage를 force
- plot_curve
 - INPUT : data, plot_type
 - OUTPUT : plot (linear, log scale)
- terminate
 - INPUT : b1500
 - Function : 장비 정상 종료

소자(트랜지스터) 하나를 측정할 때마다
Plot을 하면 실시간으로 찍히는 것을 확인할 수 있겠죠?



Parameterizing Measurement Functions

● Results and discussion

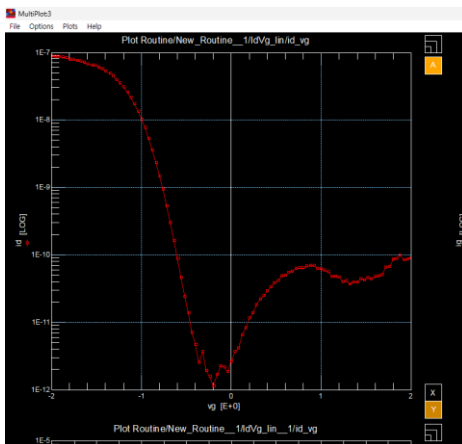
- ✓ Manual measurement (WaferPro)와 Code-based Measurement (Python) 사이의 차이가 컸음

- 현재 3개 측정 → 더 측정하면서 해결방안 모색중

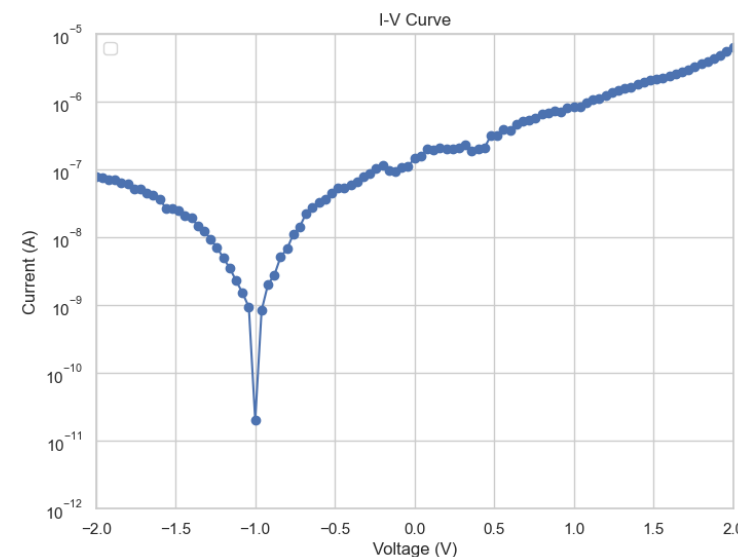
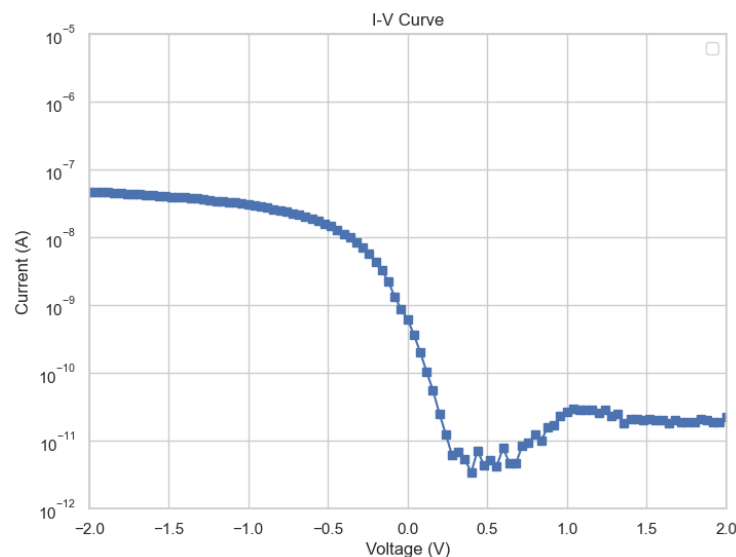
제 코드도 문제가 많았어서, 제 코드를 기반으로 새로 짰 다음에 여러 소자를 찍어보면서 보완이 필요해요.

- ✓ 현재 문제가 되는 점

- 측정 속도 : WaferPro에 비해 Python이 측정 속도가 크게 느림. WaferPro는 measurement-by-measurement 통신하는데 반해, Python은 현재 전체 measurement를 보내고 전체를 받기 때문으로 추정
 - SCPI 명령어 : Old series의 명령어들 (DV, DI 등)을 인식하지 못함 → pymeasure의 일부 함수 사용 불가, custom-func



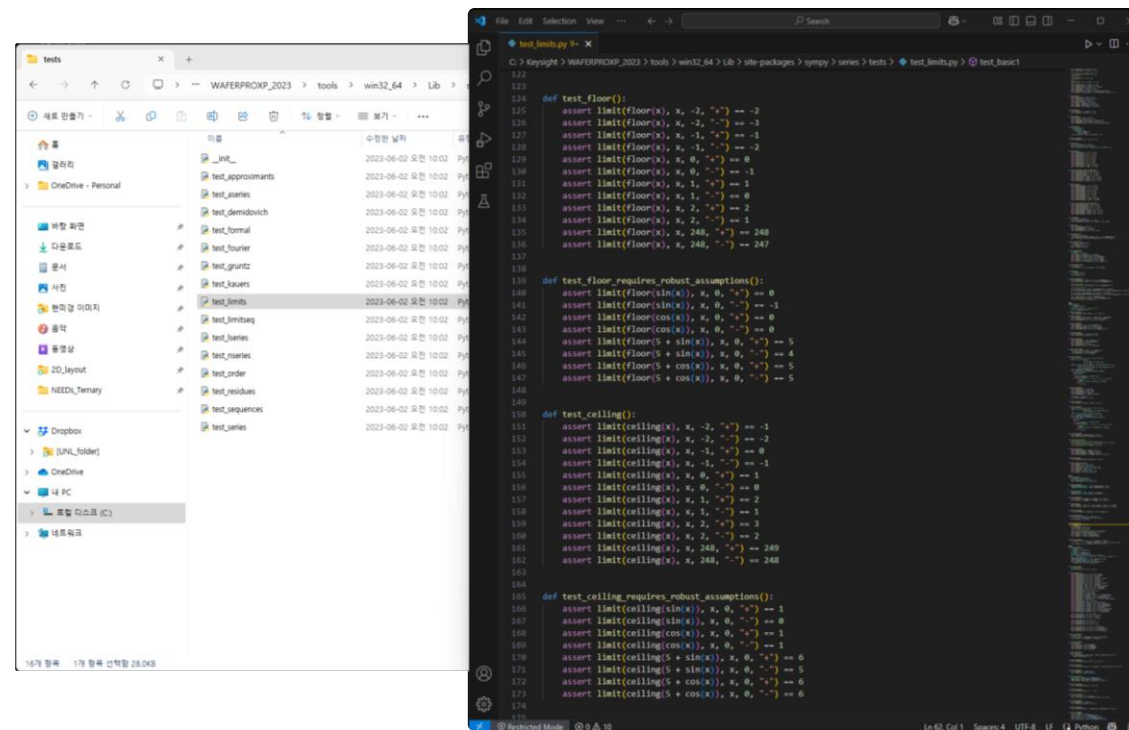
WaferPro Measurement



Code-based Measurement

● Data-based Pass/Fail 판별

- ✓ 목표 : 주어진 전압 조건 (V_{GS} , V_{DS})에 대해 소자의 전류 I_D 를 측정, Pass/Fail 판별하는 모델
- ✓ 데이터 특성
 - 전압/전류 값이 넓은 범위에 분포
 - 실제 측정값 : 비정상적인 데이터를 포함
 - Sequential한 특징을 가짐
- ✓ 기존 방식 (WaferPro)
 - Limits에 설정된 절대값 기준으로 Pass/Fail 판별
 - 다양한 물질로 만든 소자 측정에는 부적절
- ✓ 새로운 방식
 - 다양한 소자에 적절한 Pass/Fail을 도출하는 인공지능 모델 학습



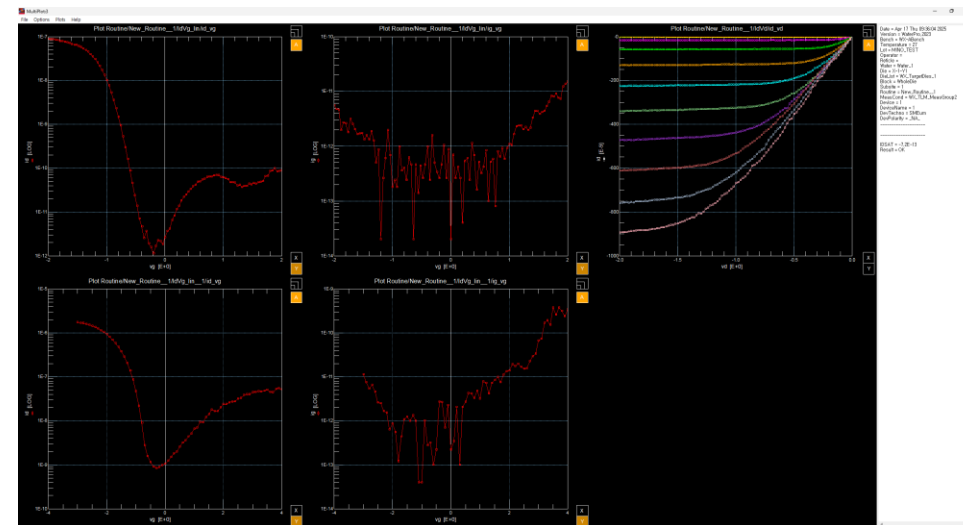
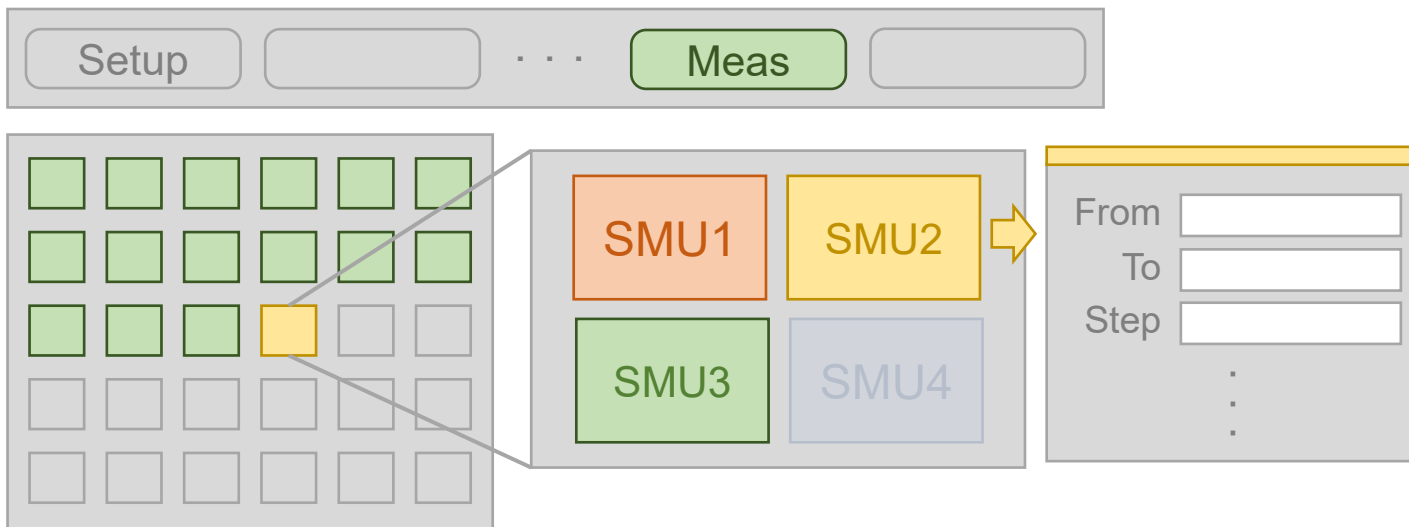
Rule-based pass/fail classification of device performance

우선 자동 측정이 잘 되면, 그 다음은 실시간으로 오류값을 배제하도록 만들면 됩니다!
여기서 팁은 결과가 이상하면 해당 측정 데이터를 아예 없애는 게 아니라 별도의 폴더나 csv 파일에 누적하도록 하면 좋아요.

Future Work

● UI/UX 이걸 예전에 측정장비 사용자에게 받았던 UI/UX 관련 의견입니다. 지금 사용자들은 다른 생각일 수 있으니 사람들에게 물어보기!

- ✓ 현재 단계 표시 : 현재 프로그램이 어떤 단계에 있는지 사용자에게 표시
- ✓ Chip
 - ✓ 지금 측정중인 소자 위치 표시
 - ✓ 각 소자에서 GUI를 통해 각 SMU를 어떤 식으로 forcing하는지 알아보기 쉽게 표시
- ✓ Plot
 - ✓ 측정 중일 때 하나의 plot에 다음 소자의 결과를 중첩해서 표시, manual하게 삭제할 수 있도록 하기
 - ✓ 특정 voltage, 특정 output level을 기준으로 abnormal한 data를 선택 후 분리된 폴더에 자동 저장/plot에서 삭제



[할 일 요약]

(1) 세미오토 제어 프로그램 만들기

SCPI를 사용해 제어하도록 신호를 보내게 하면 됨!
먼저 성민오빠에게 세미오토 사용 관련 설명을 듣고,
로직팀 승훈, 상현이나 다른 사람들과 이야기하면서 측정장비 제어에 대해 알아보기
그리고 파이썬으로 소자 측정할 수 있도록 구현하기

(2) 측정된 소자의 Pass/Fail을 실시간으로 판별할 수 있게 만들기

기존에 툴을 바탕으로 했던 거 말고
인공지능 기반으로 소자가 괜찮은지 아닌지 판별하게 하기
이걸 구현하기 위해서는 연구실에서 만드는 다양한 소자에 대한 이해가 필요하다!

[물어볼 만한 사람 리스트]

로직팀 – 승훈(CNT), 상현(MoS₂)
메모리팀 – 동진, 민호, 현진
패키징 – 형준, 이용우 박사님
RF – 성민, 경선

사실 민호선배랑 이용우 박사님께 물어보면 거의 다 커버 가능

(3) 필요한 소자에 찾아가서 측정하게 만들기 (Active Learning)

좌표평면을 쭉 측정하는 게 아니라, 조건별로 다양한 소자가 한 die안에 있으면,
이상한 결과가 나왔을 때 다음으로 측정할 소자를 알아서 찾아가게 해도 좋음
(다음 슬라이드부터 있는 reference 참고)

[공부할 만한 것]

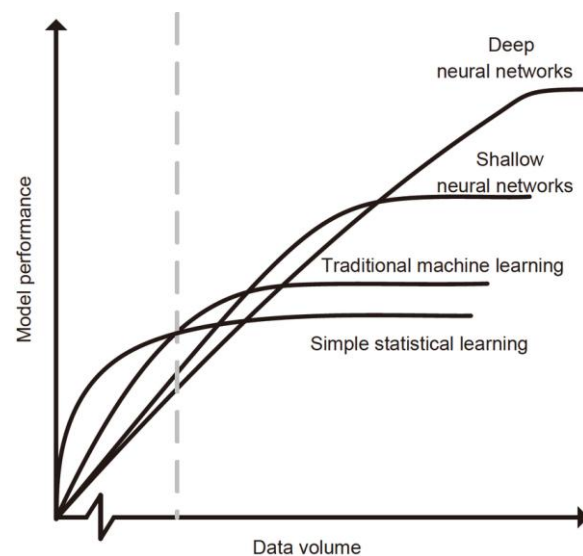
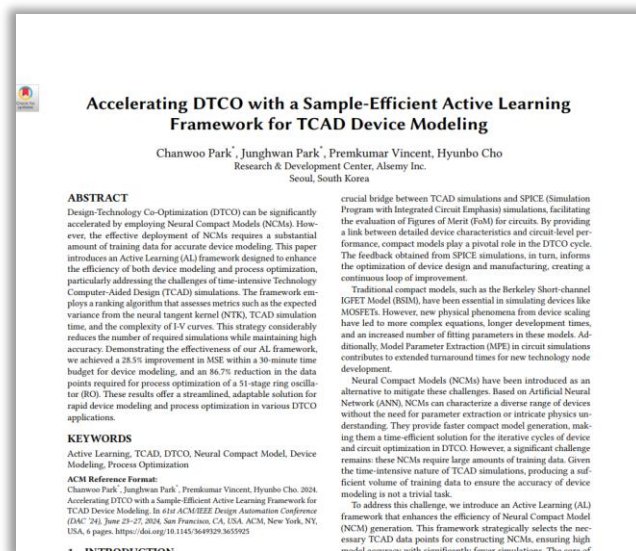
1. 측정장비 사용법 및 제어 시 주의사항
2. 파이썬 – 은 agent 도움 받아가면서 하기
3. 소자별 측정 시 경향성 (I-V, C-V)

[생각해볼 만한 것]

- 반도체 소자의 대량 측정이 왜 필요한가?

● Motivation

- ✓ Accelerating DTCO with a Sample-Efficient Active Learning Framework for TCAD Device Modeling (DAC '24)
- ✓ Neural Compact Models (NCMs) have been introduced to model more complex devices
 - However, it requires large amounts of training data
 - Given the time-intensive nature of TCAD simulations, producing a sufficient volume of training data to ensure the accuracy of device modeling is not a trivial task
- ✓ Active Learning (AL) framework to enhance the efficiency of NCM generation.



Jin et al., Sci. China Earth Sci. 63 (3), 2020

What to simulate in TCAD?



Ranking System under 3 conditions

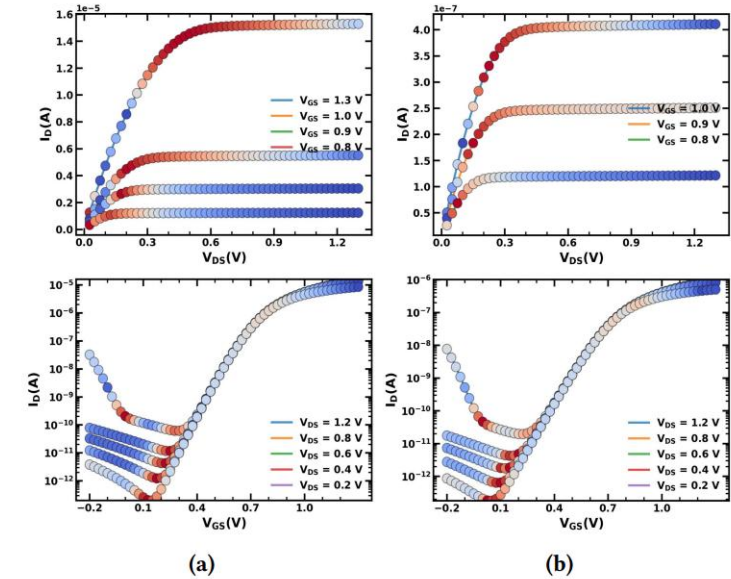
1. Function Complexity m_c
2. Expected Variance m_{ev}
3. Simulation Time m_t

Paper Review : Active Learning (AL) Framework

● Function Complexity : Curvature + Pre-trained NN

✓ Curvature κ

$$\kappa = \frac{\frac{d^2 y}{dx^2}}{\left(1 + \left(\frac{dy}{dx}\right)^2\right)^{3/2}}$$



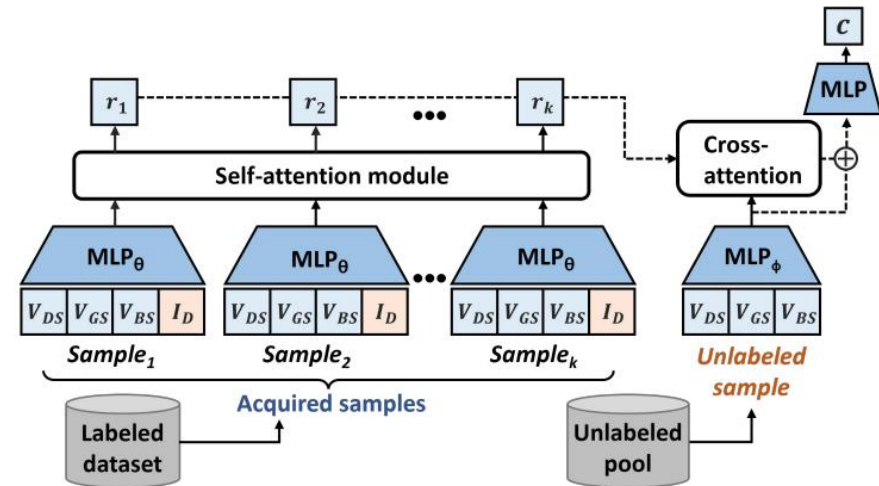
- I_D for y
- V_{DS} , V_{GS} , and V_{BS} for x

→ larger κ means more complex part

✓ How to extract function complexity? → Pre-trained NN

- Transformer based encoder-decoder + MLP

1. Self-attention
2. Cross-attention
3. MLP



Paper Review : Active Learning (AL) Framework

- **Gaussian Process (GP) with Neural Tangent Kernel (NTK)**

- ✓ Gaussian Process (GP) : used to calculate expected variance

$$m_{ev} = -\frac{1}{|U|} \sum_{x \in U} \sigma_{GPNTK}^2(x|L \cup \{x_i\})$$

the amount of variance reduction if we simulate x_i in addition to L

→ larger m_{ev} means meaningful data point

- ✓ Neural Tangent Kernel (NTK)
 - A theoretical tool that enables kernel-based analysis of the learning and prediction of complex neural networks.
 - Simply put, it allows us to interpret deep learning models as linearized kernel models
 - Used for GP's kernel

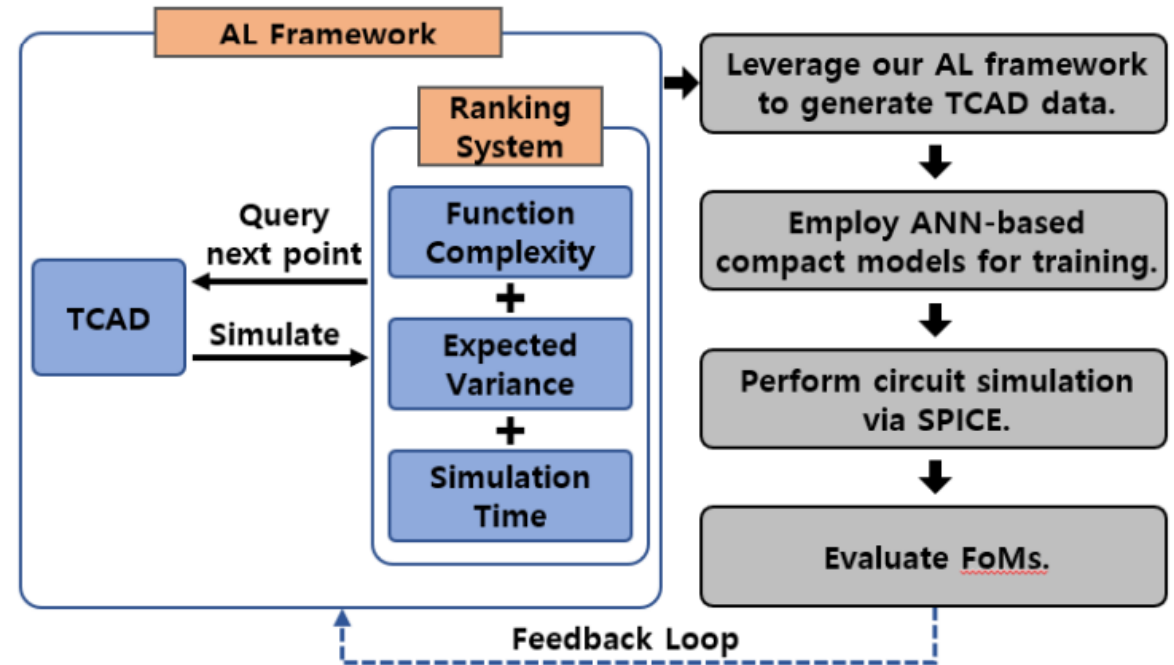
Paper Review : Active Learning (AL) Framework

● Active Learning (AL) framework

- ✓ For each batch of iterations, calculate the score based on m_c , m_t , and m_{ev} .
- ✓ Simulate the point that is most meaningful (gives the most information to the model)
- ✓ Iterate until error is less than the target spec

Algorithm 1 Active Learning Procedure

```
1: Input: Initial labeled data  $\mathcal{L}$ , validation data  $\mathcal{V}$ , unlabeled data  $\mathcal{U}$ , query size  $q$ , function complexity estimator  $f_c$ , simulation time estimator  $f_t$ , GP with NTK ( $GP_{\text{NTK}}$ )
2: while error > target do
3:   for each batch of  $q$  iterations do
4:     for  $i$  from 1 to  $|\mathcal{U}|$  do
5:        $m_c = f_c(x_i|\mathcal{L})$ ;
6:        $m_t = f_t(x_i)$ ;
7:        $m_{ev} = -\frac{1}{|\mathcal{U}|} \sum_{x \in \mathcal{U}} \sigma_{GP_{\text{NTK}}}^2(x|\mathcal{L} \cup \{x_i\})$ ;
8:       Compute the aggregated score:
9:        $\text{score}_i = w_c \cdot m_c + w_t \cdot m_t + w_{ev} \cdot m_{ev}$ ;
10:    end for
11:     $x^* = \arg \max_{x \in \mathcal{U}} \{\text{score}_i\}_{i=1}^{|\mathcal{U}|}$ ;
12:     $\mathcal{L} = \mathcal{L} \cup \{(x^*, y^*)\}$ ,  $\mathcal{U} = \mathcal{U} \setminus \{x^*\}$ ;
13:  end for
14:  error =  $\frac{1}{|\mathcal{V}|} \sum_{(x,y) \in \mathcal{V}} (GP_{\text{NTK}}(x|\mathcal{L}) - y)^2$ 
15: end while
```



Paper Review : Active Learning (AL) Framework

● Experimental Results and Discussion

- ✓ Labeled data의 양에 비해 Validation data에 대한 Mean Squared Error (MSE) ↓
- ✓ Timing budget (30 min. & 60 min.) 내에서 가장 좋은 MSE 기록
- ✓ Key Points
 - Sample Efficiency
 - Time Efficiency

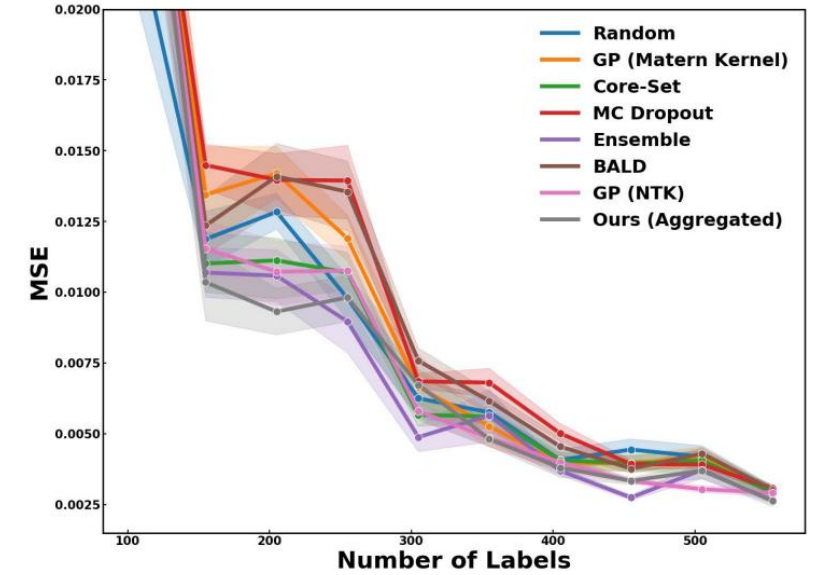


Table 1: TCAD Parameters for Device Modeling.

Parameters	Value
Length	45 nm
Gate oxide thickness	1 nm
Gate electrode material	Poly-Si
Gate electrode thickness	100 nm
Boron doping	3.24×10^{18}
Arsenic doping	$8 \times 10^{20} \text{ cm}^{-3}$
Doping profile	Gaussian
Junction depth (X_j)	14 nm
Extension junction depth	$X_j * 0.4$
Source/Drain electrode length	50 nm
Substrate thickness	300 nm

Method	30 min. Budget				60 min. Budget			
	TCAD (sec)	Training (sec)	# Data	MSE	TCAD (sec)	Training (sec)	# Data	MSE
Random	1,772	N/A	205	1.3	3,583	N/A	425	0.35
GP (Matern Kernel)	1,727	1.30	205	1.4	3,543	4.00	425	0.37
BALD [14]	1,566	173	185	1.4	2,862	666	345	0.75
MC Dropout [3]	1,561	174	185	1.3	2,864	669	345	0.72
Ensemble [2]	1,409	361	165	1.2	2,218	1,084	265	0.65
Core-Set [15]	1,750	N/A	205	1.1	3,567	N/A	425	0.40
GP (NTK) [4]	1,739	10.1	205	1.1	3,562	20.9	425	0.35
Ours (Aggregated)	1,752	10.3	205	0.93	3,570	21.4	425	0.31

Remaining Challenges

● Remaining Challenges and Research Area

✓ Lack of Real Measurement Data and Practical Constraints

- TCAD 기반 가상 데이터에 의존

→ measurement data 기반, AL framework를 적용하여 NCM 학습

✓ Static and Simulation-Centric Active Learning Strategy

- Aggregated Ranking은 고정된 가중치와 시뮬레이션 시간만을 고려

→ 실험적인 고정 가중치 ($w_c:w_t:w_{ev} = 5:-1:2$)를 사용한 이유는 시뮬레이션상 (Heuristic Search) 가장 효과적인 비율

→ 실제 측정 결과를 가지고 학습시킬 때에는 다를 수 있다

✓ Explainability of AI Models

→ Park et al., *IEEE J. Emerg. Sel. Top. Circuits Syst.* 12, 2024

